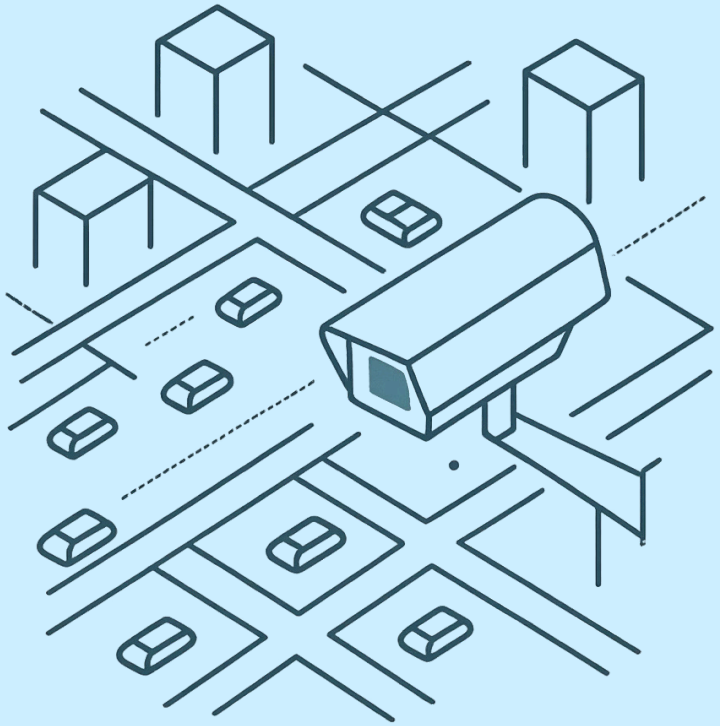
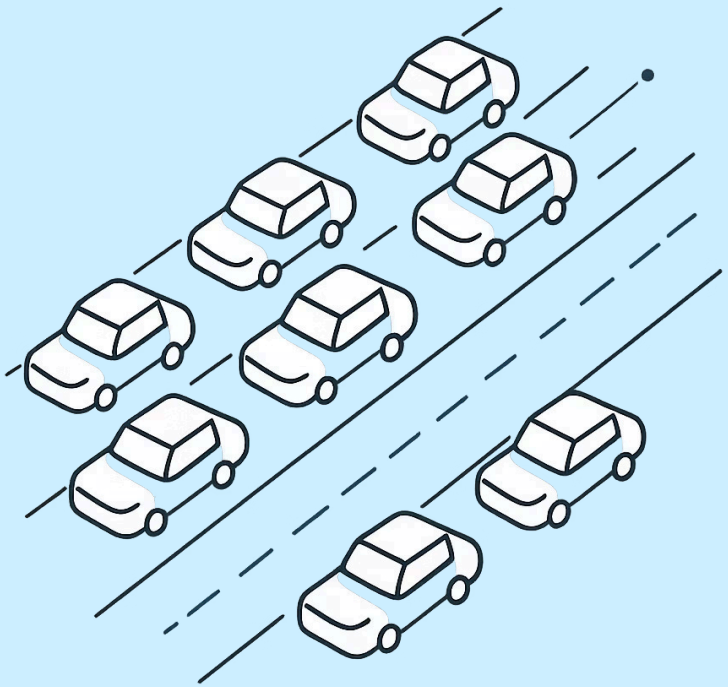
Traffic Density Estimation Using CSRNet

Piyansu Bagaria · Anubhab Basu · Aatman Gupta

DEPT. OF CSE, MIT MANIPAL, INDIA





Traffic Congestion 4 A Growing Urban Crisis

The Problem

Rapid vehiclegrowth causes severe congestion 4 increasing travel time, fuel waste, and emissions across modern cities.

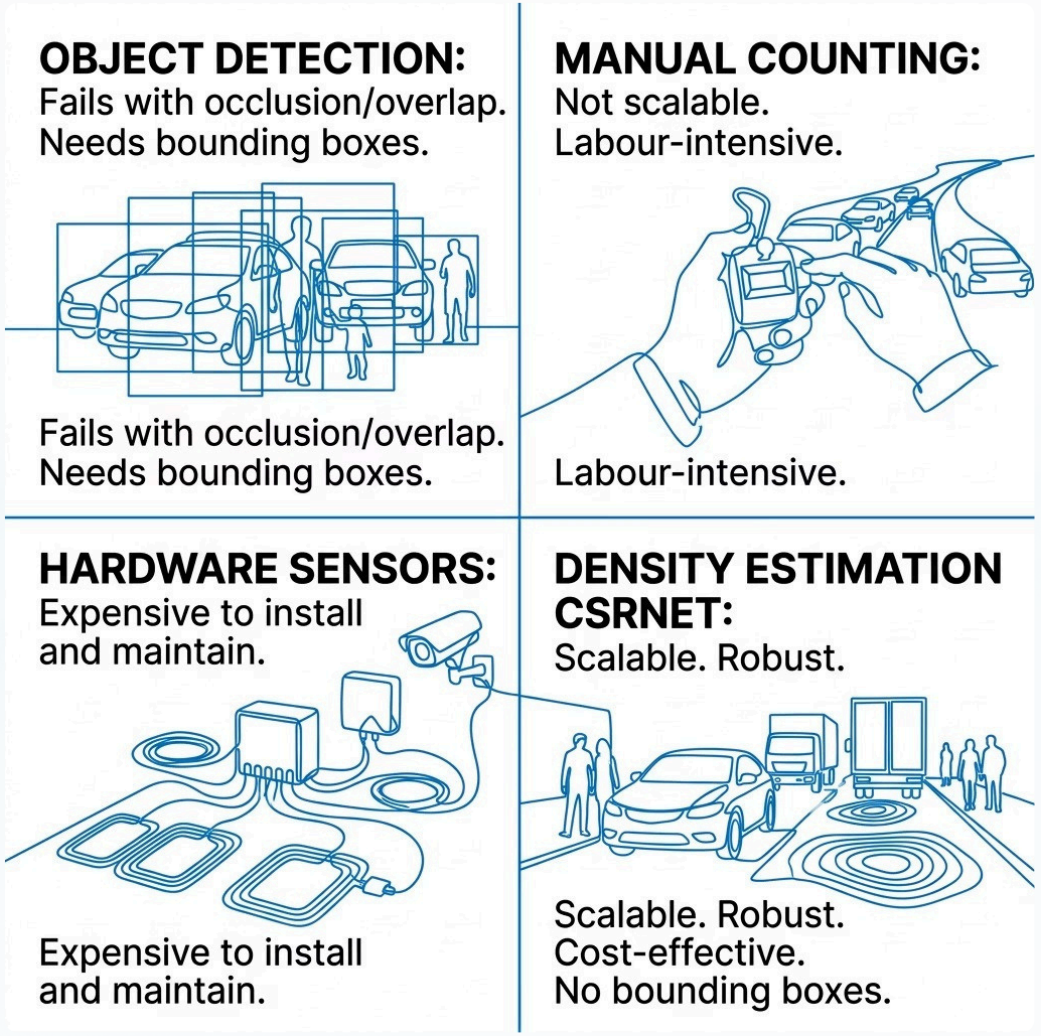
Traditional Methods

Hardware sensors and manual counting are expensive, labour-intensive, and not scalable for city-wide deployment.

The Need

Anintelligent,automated, and cost-effective solution leveraging existing surveillance infrastructure.

Why Density Estimation Over Object Detection?



Object detection fails under occlusion and heavy overlap. Manual counting does not scale. Hardware sensors are costly to deploy and maintain. **CSRNet density estimation** is the only approach that is scalable, robust, and cost-effective 4 making it ideal for real-world traffic surveillance.

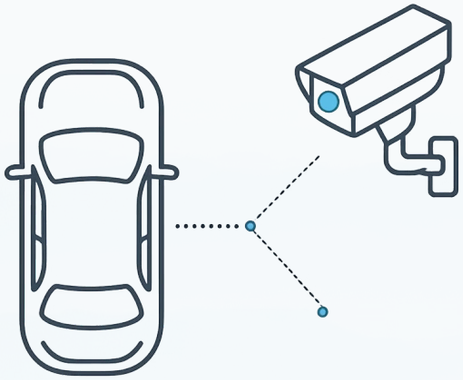
Project Objective & End-to-End Pipeline

Predicta continuousdensitymapfromsurveillance images and computetotalvehiclecount:**Count=£D(x)** across all pixel locations. Worksreliablyunderocclusion,varying lighting, and heavycongestion.



The pipeline transforms raw surveillance frames into accurate vehicle counts through learned density regression 4 no bounding boxes required at any stage.

Dataset 4 TRANCOS



Dataset Overview

TRANCOS isa publiclyavailable traffic surveillance dataset with diverse scenarios: low and heavy traffic, day and night conditions, and significant occlusions.

- **Train:** 1,031 images
- **Test:** 213 images
- **Annotations:** Dot labels per vehicle
- **Mask:** Road-region binary mask included

```
DATA_ROOT = "TRANCOS - edited"

train_ds = TrancosDataset(
 os.path.join(DATA_ROOT, "train_data"),
 downsample_factor=2
)
test_ds = TrancosDataset(
 os.path.join(DATA_ROOT, "test_data"),
 downsample_factor=2
)
print("Train:", len(train_ds)) # 1031
print("Test:", len(test_ds)) # 213
```

Data Preprocessing & Density Map Generation

Image Preprocessing

Resize, normalise (ImageNet mean/std), and apply road mask to focus only on the road region. Augmentation includes flip, crop, and scale.

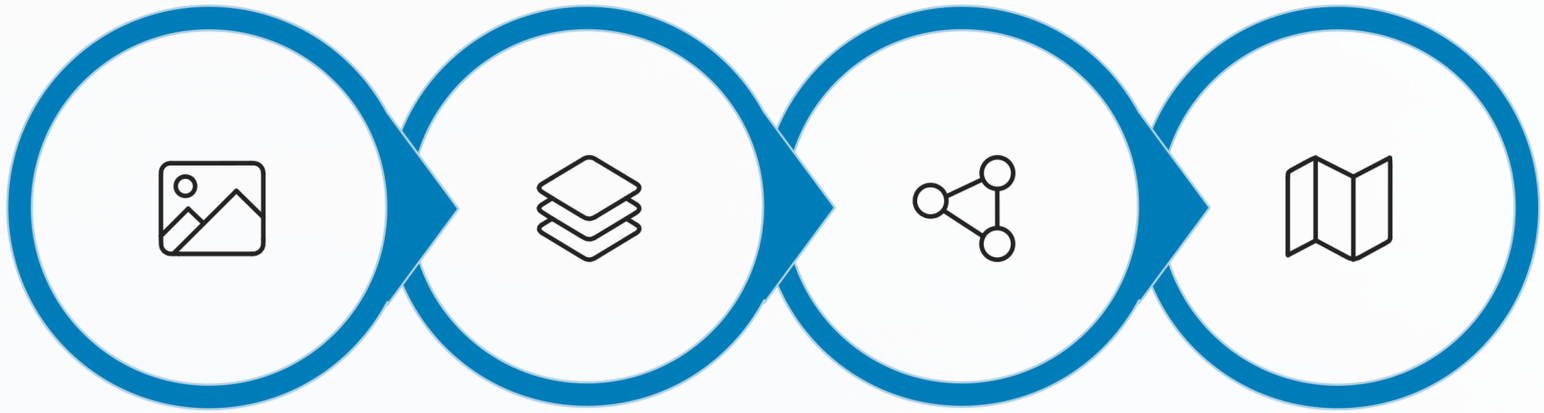
```
self.transform = transforms.Compose([
transforms.ToTensor(),
transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
# Apply road mask
img = img * mask[:, :, None]
```

Gaussian Density Map

Each vehicledotisconvolved with a 2D Gaussian kernel (Ã=4). Key property: **+ D(x) dx = N** (total count).

```
def generate_density_map(shape, points, sigma=4):
H, W = shape
density = np.zeros((H, W), dtype=np.float32)
for x, y in points:
 x, y = int(x), int(y)
 if x >= W or y >= H: continue
 density[y, x] += 1
 density = cv2.GaussianBlur(
density, (15, 15), sigma
)
return density
```

CSRNet Architecture



Input Image

Frontend VGG-16

Backend Dilated

Density Map

The frontend extracts rich visual features using pretrained VGG-16, while the backend uses dilated convolutions to expand the receptive field without downsampling 4 preserving spatial resolution for accurate density regression.

Frontend 4 VGG-16

First 13 conv layers, pretrained on ImageNet. Outputs 512-channel feature maps.

Backend 4 Dilated Convs

$512^3512^3256^3128^364$ with dilation rate=2. Expands receptive field without losing resolution.

Output Layer

1×1 convolution produces a single-channel density map D(x).

Model Implementation & Training

CSRNet Model

```
class CSRNet(nn.Module):
    def __init__(self):
        super().__init__()
        vgg = models.vgg16(
            weights=models.VGG16_Weights.DEFAULT
        ).features
        self.frontend = nn.Sequential(
            *list(vgg.children())[:23]
        )
        self.backend = nn.Sequential(
            nn.Conv2d(512,512,3,padding=2,dilation=2),
            nn.ReLU(),
            nn.Conv2d(512,256,3,padding=2,dilation=2),
            nn.ReLU(),
            nn.Conv2d(256,128,3,padding=2,dilation=2),
            nn.ReLU(),
            nn.Conv2d(128,64,3,padding=2,dilation=2),
            nn.ReLU()
        )
        self.output = nn.Conv2d(64, 1, 1)

    def forward(self, x):
        x = self.frontend(x)
        x = self.backend(x)
        return self.output(x)
```

Training Loop

```
device = torch.device(
"cuda" if torch.cuda.is_available() else "cpu"
)
model = CSRNet().to(device)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(
 model.parameters(), lr=5e-5
)
for epoch in range(10):
 for img, density in train_loader:
img, density = img.to(device), density.to(device)
pred = model(img)
loss = criterion(pred, density)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Vehicle Count Estimation & Evaluation

Count from Density Map

```
# Predict density map
pred_map = model(image)

# Sum all pixels ³ vehicle count
vehicle_count = pred_map.sum().item()
print(f"Estimated: {vehicle_count:.1f}")
```

Bright regions = high density. No bounding boxes needed. Works under heavy occlusion.

Evaluation Metrics

```
mae = np.mean(np.abs(pred_counts - gt_counts))
mse = np.sqrt(
np.mean((pred_counts - gt_counts)**2)
)
print(f"MAE: {mae:.2f}")
print(f"MSE: {mse:.2f}")
```

MAE	Average absolute count error
MSE	Sensitivity to large errors

Tools, Future Scope & Conclusion

TechStack

Py thon	Core language
PyTorch	Model training & inference
OpenCV	Image processing & Gaussian blur
NumPy	Array & density map ops
Matplotlib	Loss plots & visualisation
SciPy	Loading MATLAB mask files

Future Directions

→ Real-Time Deployment

Live CCTV feed integration

→ Smart Signal Control

Traffic signal integration

→ Edge Deployment

Jetson Nano, RaspberryPi

Conclusion: CSRNet estimates traffic density from surveillance images, handling occlusion and varying conditions 4 a strong foundation for Intelligent Transportation Systems. *"Smarter roads start with smarter sensing."*